

Check Point 01

Ferramentas open source de SAST, DAST,
IaC Security e SCA no pipeline

3

grupos

4

categorias cada

12

ferramentas no total

30

minutos por grupo

Pesquisa em grupo + apresentação + laboratório prático conduzido para a turma

Professor: _____ Turma: _____ Apresentações: ____/____/____

O que já vimos e o que ainda falta

Módulo II

GitLeaks

detecção de segredos em repositórios e no histórico Git

Bandit

análise estática (SAST) restrita a Python

Duas ferramentas úteis — e um recorte muito pequeno do problema.



Check Point 01

Quatro frentes complementares de teste automatizado, cobertas por 12 ferramentas open source diferentes ao longo das três apresentações.

SAST

SCA

IaC Security

DAST

A confusão mais comum da disciplina: SAST não é SCA. SAST olha o código que a sua equipe escreveu. SCA olha as dependências de terceiros que você trouxe para dentro. São problemas diferentes, com ferramentas diferentes e com formas de correção diferentes.

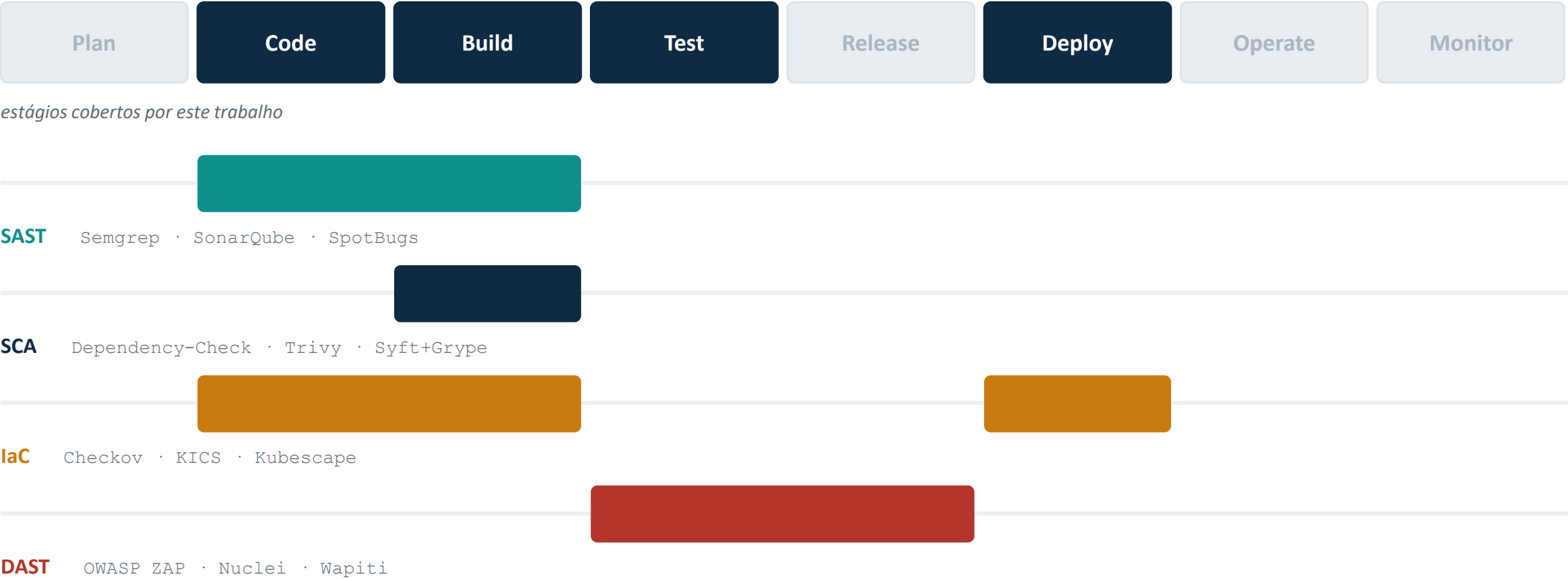
O E | CDE cobre segurança nos estágios plan, code, build, test, release, deploy, operate e monitor. Este trabalho ataca code, build, test e deploy.

As quatro frentes que o pipeline precisa cobrir

SAST Static Application Security Testing	SCA Software Composition Analysis	IaC Infrastructure as Code Security	DAST Dynamic Application Security Testing
ANALISA Código-fonte próprio (white-box)	ANALISA Dependências de terceiros e licenças	ANALISA Terraform, CloudFormation, K8s, Docker, Helm	ANALISA Aplicação em execução (black-box)
ESTÁGIO Code / Build	ESTÁGIO Build	ESTÁGIO Code / Build / Deploy	ESTÁGIO Test / Release
APP RODANDO? Não	APP RODANDO? Não	APP RODANDO? Não	APP RODANDO? Sim

Shift-left não elimina o DAST. SAST, SCA e IaC olham artefatos parados e não veem erro de configuração de runtime, falha de autenticação de sessão nem comportamento da aplicação integrada. Por isso o DAST continua no pipeline — e por isso ele é o único que exige a aplicação no ar.

Onde cada categoria entra no pipeline



IaC aparece duas vezes de propósito. O mesmo manifesto é analisado quando é escrito e verificado de novo antes de provisionar. Um Terraform aprovado no commit pode virar um recurso inseguro no deploy se a variável de ambiente mudar.

Como os grupos se organizam

1

Líder técnico

Responsável pelo repositório e pelo ambiente Docker. É quem garante que a turma consegue rodar o laboratório.

2

Relator

Responsável pelo documento de pesquisa e pela consistência entre o que está escrito e o que foi demonstrado.

3

Todos apresentam

Cada integrante apresenta uma parte do conteúdo. Quem não apresenta não recebe a nota de apresentação.

4

Repositório único

Documento, slides, roteiro do laboratório, código e evidências. Público ou compartilhado com o professor.

O histórico de commits é avaliado. Repositório com um único commit final feito por uma pessoa não caracteriza trabalho em grupo, por mais completo que esteja o conteúdo.

As 12 ferramentas, três por categoria

	Grupo 1 Toolchain OWASP Pipeline tradicional	Grupo 2 Cloud native Containers	Grupo 3 Supply chain, SBOM e Kubernetes
SAST	Semgrep	SonarQube CE	SpotBugs + FindSecBugs
SCA	OWASP Dependency-Check	Trivy	Syft + Grype
IaC	Checkov	KICS	Kubescape
DAST	OWASP ZAP	Nuclei	Wapiti

Substituição só com autorização prévia. A ferramenta precisa ser open source, ter commit nos últimos 12 meses e não coincidir com a de outro grupo. O banco de alternativas está no Anexo C do documento.

O que cada grupo entrega

01

25 pts

Documento de pesquisa

- PDF, 15 a 25 páginas de conteúdo
- Fonte 11 ou 12, espaçamento 1,5, ABNT
- Mínimo 10 fontes, 4 delas primárias
- Responde ao roteiro da seção 6

02

20 pts

Apresentação

- .pptx e .pdf de segurança
- 20 a 30 slides, máximo 30 minutos
- Capa, agenda, fundamentação, demo
- Quadro comparativo e limitações

03

35 pts

Laboratório conduzido

- Mínimo 2 ferramentas aplicadas
- De categorias diferentes
- Executado ao vivo pela turma inteira
- É o entregável que mais vale

Os 20 pontos restantes

- 10 pts — repositório e artefatos: organização, README, commits distribuídos, relatórios de saída versionados
- 10 pts — participação individual: executar os laboratórios dos outros dois grupos e entregar o comprovante

O laboratório conduzido — 7 requisitos

1

Roteiro escrito

LAB.md com pré-requisitos, passos numerados, comandos copiáveis e resultado esperado de cada passo

3

Publicação 24h antes

A turma precisa fazer git clone e docker pull antes da aula. Sem isso, -10 pontos

5

Pipeline com gate

GitHub Actions ou GitLab CI quebrando o build em HIGH/CRITICAL. Mostrar build vermelho e verde

7

Análise de 3 achados

Verdadeiro ou falso positivo, com justificativa técnica, CWE associado e correção proposta

2

Ambiente reproduzível

docker-compose.yml ou Dockerfile que suba tudo. Nada de "instala aí e boa sorte"

4

Alvo vulnerável conhecido

Da lista autorizada, para que as ferramentas encontrem achados reais

6

Plano B gravado

Vídeo de 5 a 8 minutos ou GIF da execução completa. Sem plano B, o risco é do grupo

As 2 ferramentas aplicadas têm que ser de categorias diferentes. Um SAST mais um DAST vale; dois SAST não valem. A ideia é que a turma veja o contraste entre analisar código parado e atacar a aplicação no ar.

Quem assiste também põe a mão na massa

1

Antes da aula

Clonar o repositório do grupo apresentador e baixar as imagens Docker. O repositório sai 24h antes, justamente para isso.

2

Durante a apresentação

Executar o laboratório na própria máquina, acompanhando o roteiro conduzido pelo grupo.

3

Ao final da aula

Entregar individualmente o comprovante de execução: print do output final e a resposta às 2 perguntas de verificação definidas pelo grupo.

Exemplo de pergunta de verificação

"Quantos achados de severidade HIGH o scanner reportou no arquivo X?"

A pergunta só é respondível por quem realmente executou. É isso que separa assistir de participar — e é isso que garante que as três apresentações valham para a turma inteira, e não só para quem apresentou.

Consequência prática para quem apresenta: o laboratório precisa caber em 12 minutos e funcionar em máquina alheia. Testem em um computador que não seja o de vocês antes de publicar.

Como distribuir os 30 minutos

3 min	Abertura e posicionamento no pipeline	Onde cada categoria entra no ciclo DevSecOps
10 min	As 4 ferramentas	Cerca de 2,5 min cada: o que é, como funciona, o que detecta
12 min	Laboratório guiado com a turma	A turma executa as 2 ferramentas escolhidas
3 min	Comparativo, limitações e conclusão	Quadro comparativo e recomendação de toolchain
2 min	Perguntas	

O bloco do laboratório é o maior da apresentação — 12 dos 30 minutos. Ensaíem cronometrado: em máquina alheia, com rede da instituição, tudo demora mais do que no ensaio de casa. Desconto de 2 pontos por minuto excedido.

O que o documento precisa responder

a

Identificação

Mantenedor, licença, ano, linguagem. Atividade do projeto: commits, última release, comunidade.

b

Fundamento técnico

Que técnica usa (taint analysis, AST, pattern matching, fuzzing...). O que detecta e o que NÃO detecta.

c

Instalação e uso

Formas de instalação, comandos e flags, formatos de saída, regras próprias e supressão de falso positivo.

d

Integração

CI/CD, IDE e pre-commit. Suporta SARIF? Integra com Code Scanning ou DefectDojo?

e

Avaliação crítica

Falsos positivos observados, tempo de execução, limitações, cenário ideal e por que escolher esta.

f

Seções transversais

SAST vs SCA, posicionamento no pipeline, shift-left, SBOM e quadro comparativo final.

"Análise crítica própria" vale 5 pontos e não é resumo de documentação. Quero a taxa de falso positivo que VOCÊS observaram, o tempo que a ferramenta levou no projeto de VOCÊS e a limitação que VOCÊS encontraram na prática.

Onde vocês podem apontar as ferramentas

Aplicações web

SAST · SCA · DAST

- OWASP Juice Shop (Node.js)
- OWASP WebGoat (Java)
- OWASP NodeGoat (Node.js)
- DVWA (PHP)
- DSVW — Damn Small Vulnerable Web (Python)

Infraestrutura como código

IaC Security

- TerraGoat (Terraform)
- CFNGoat (CloudFormation)
- KubernetesGoat
- Manifests deliberadamente inseguros

Estes alvos existem para serem quebrados. São aplicações mantidas pela OWASP e pela comunidade justamente para treinamento — por isso as ferramentas vão encontrar achados reais, e não uma tela verde sem graça. Aplicações próprias do grupo também são aceitas.

Não negociável

Só os alvos da lista, e só em ambiente isolado

É terminantemente proibido executar DAST, fuzzing ou qualquer varredura ativa contra sistemas de terceiros, sites públicos ou o ambiente da empresa onde você trabalha. Docker ou VM local, preferencialmente sem exposição à rede externa.

Varredura sem autorização é crime no Brasil

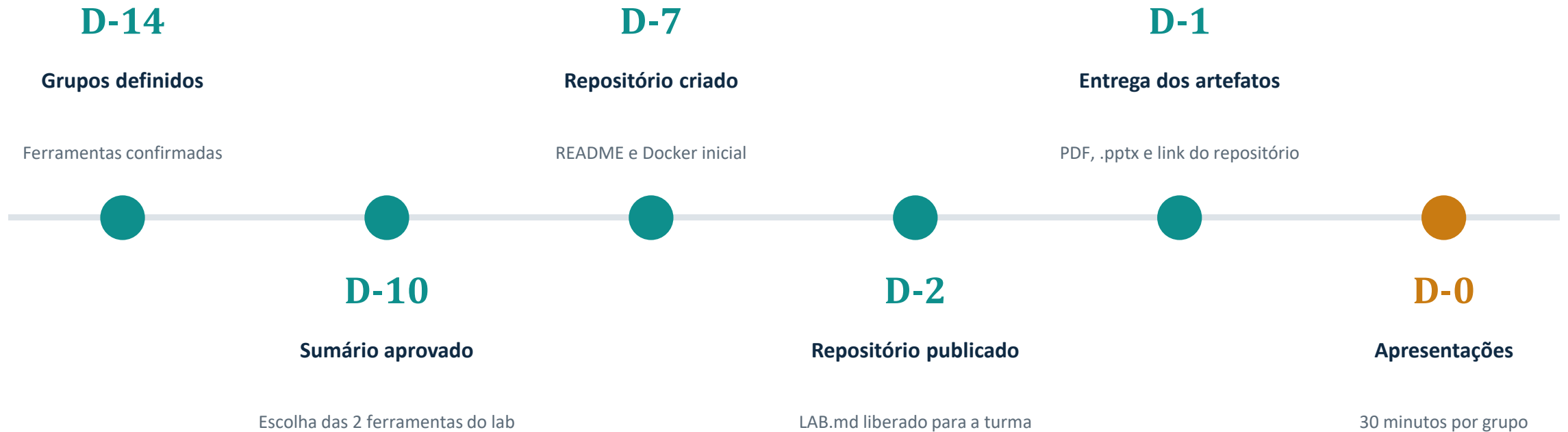
Lei 12.737/2012 e artigo 154-A do Código Penal. Não existe "eu só estava testando": o que caracteriza o crime é o acesso sem autorização, não a intenção declarada depois.

A consequência está escrita no enunciado

Trabalho que apresente evidência de varredura em alvo não autorizado recebe nota zero e o caso é encaminhado à coordenação.

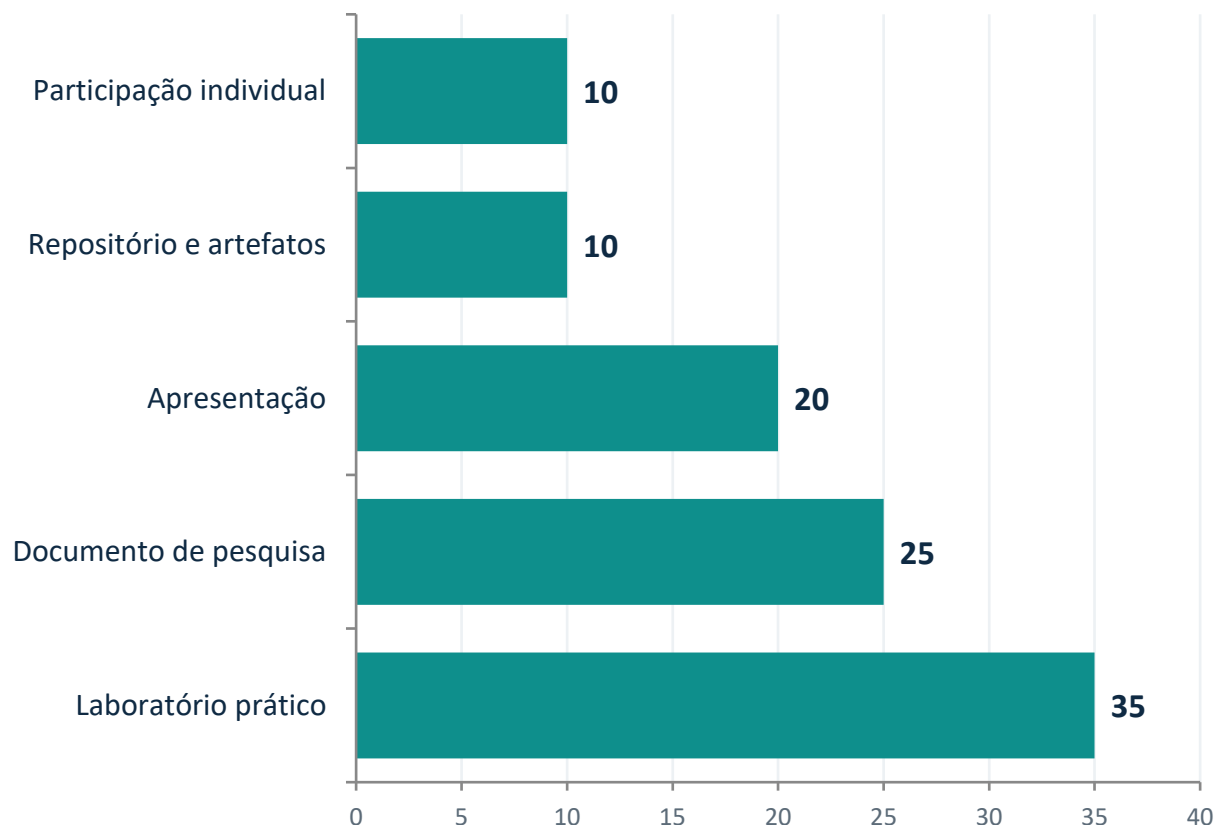
O DAST é a única categoria deste trabalho que ataca um alvo em execução. É por isso que a regra aparece aqui e não no módulo passado.

Cronograma



O marco D-2 é o mais crítico do cronograma. É quando o repositório e o LAB.md ficam disponíveis para a turma preparar o ambiente. Atrasar esse marco significa laboratório que não roda no dia — e desconto de 10 pontos.

Como a nota é composta



Laboratório (35)

Ambiente reproduzível 10 · Roteiro LAB.md 6 · Pipeline com gate 8 · Análise dos 3 achados 8 · Plano B 3

Documento (25)

Cobertura do roteiro 10 · Profundidade técnica 7 · Análise crítica própria 5 · ABNT 3

Apresentação (20)

Clareza e domínio 8 · Slides 4 · Tempo 4 · Participação equilibrada 4

Repositório (10)

Organização e commits 5 · Relatórios versionados 5

Individual (10)

Executar os labs dos outros dois grupos + comprovante 10

O prático pesa 45 dos 100 pontos (laboratório 35 + participação individual 10). Este é um trabalho de execução com fundamentação, não uma pesquisa bibliográfica com demonstração no final.

Regras de conduta acadêmica

Citação obrigatória

Todo trecho, imagem, diagrama ou comando copiado de terceiros deve ser referenciado. Plágio anula o trabalho.

IA generativa é permitida e incentivada

Desde que declarada em um anexo USO-DE-IA.md indicando o que foi gerado e como foi validado. O que não é aceitável é entregar texto gerado sem verificação: informação técnica incorreta sobre a ferramenta é penalizada independentemente da origem.

Toda afirmação técnica precisa ser verificável

Se o documento diz que a ferramenta detecta SQL injection em PHP, tem que existir o relatório versionado que comprova. Afirmação sem evidência é afirmação que não conta.

Qualquer integrante pode ser questionado sobre qualquer parte do trabalho na hora da apresentação. A divisão interna de tarefas não isenta ninguém de compreender o conjunto.

Por onde começar hoje

- 1 Formar os 3 grupos e eleger líder técnico e relator
- 2 Confirmar as 4 ferramentas do grupo (ou pedir substituição autorizada)
- 3 Escolher as 2 ferramentas do laboratório — de categorias diferentes
- 4 Criar o repositório e subir o esqueleto do docker-compose.yml
- 5 Escolher o alvo vulnerável e conferir que ele gera achados de verdade

O enunciado completo — roteiro do documento, template do LAB.md, checklist de entrega e banco de ferramentas alternativas — está no arquivo Word do Check Point 01.